

Name : Aryan Chaudhari

Roll No: 25MCA009

Practical – 2 : Docker

(MAP)

PART – 1

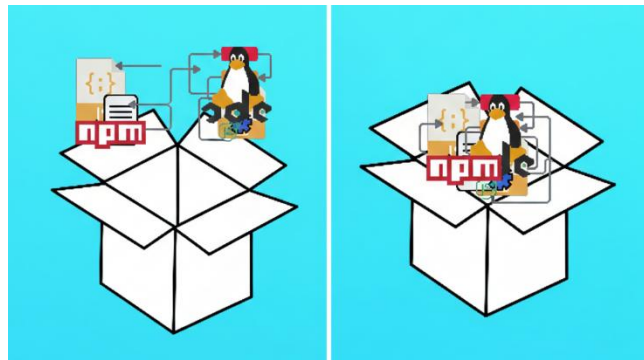
1. What is Docker & Why?

Definition:

Virtualisation software

Makes developing and deploying applications much easier

Docker is a **containerization platform** that allows developers to package an application and its dependencies into a single unit called a container, ensuring consistent execution across different environments.

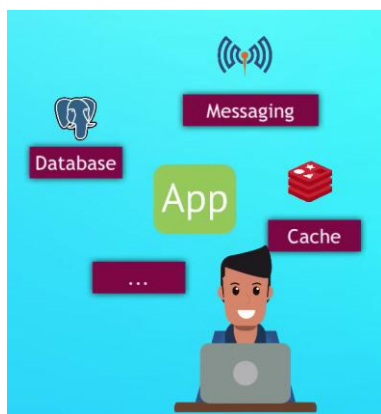


Let's See How Containers Solve Some of the Problems ??

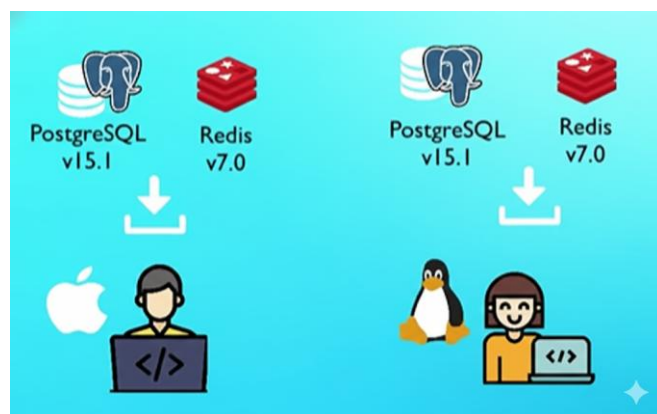
Problem:

Each developer needs to install and configure all services directly on their OS on their local machine.

SOLO



Multiple workers working



➔ Installation process different for each OS Environment

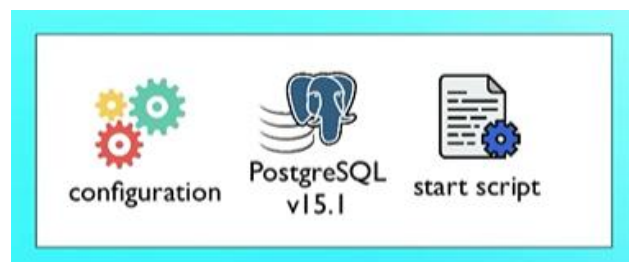
➔ Many steps, where something can go wrong

If your app uses 10 services, each developer needs to install all 10 services.



Solution using Docker:

- Own isolated environment
- Postgres packaged with all dependencies and configs

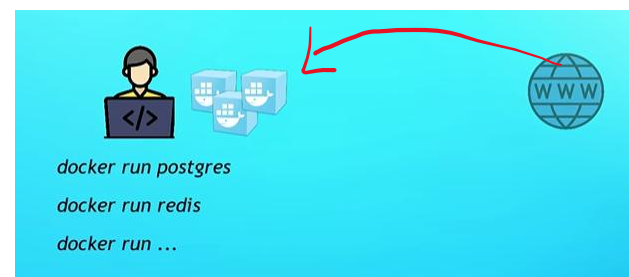


(Start service as a Docker container using a 1 docker command)

- Command same for all OS



- Command same for all services
- Standardised process of running any service on my local dev environment



- Easy to run different version of same app without any conflicts



Development Process Before Containers??

- Applications were deployed on physical servers or virtual machines.
- OS, runtime, libraries, and dependencies were installed manually.
- Deployment was slow and caused “*works on my machine*” problems.
- Scaling required setting up new servers or VMs, which took time and cost more.

Problems:

- Installations and configurations done directly on the server’s OS
- Dependency version conflicts, etc.
- Deployment and scaling were slow due to manual server setup.
- Resources were not used efficiently, and rollback was difficult.

What Solution we get by Docker?

- No configurations needed on the server (except Docker runtime)
- Less room for errors
- Install Docker runtime on the server
- Run docker command to fetch and run the docker artifacts



2. Virtual Machine vs Docker

To understand this, let's understand how docker work on technical level 😊:

If you use docker you don't need to install services directly on the operating system.

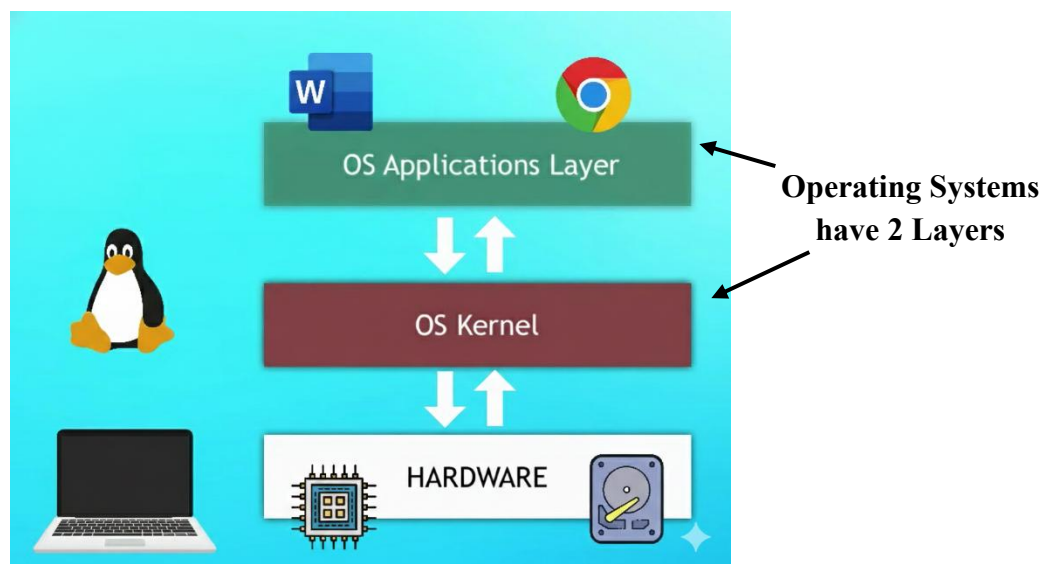
Then how does Docker run its containers in an operating system?

(Let's first understand OS)

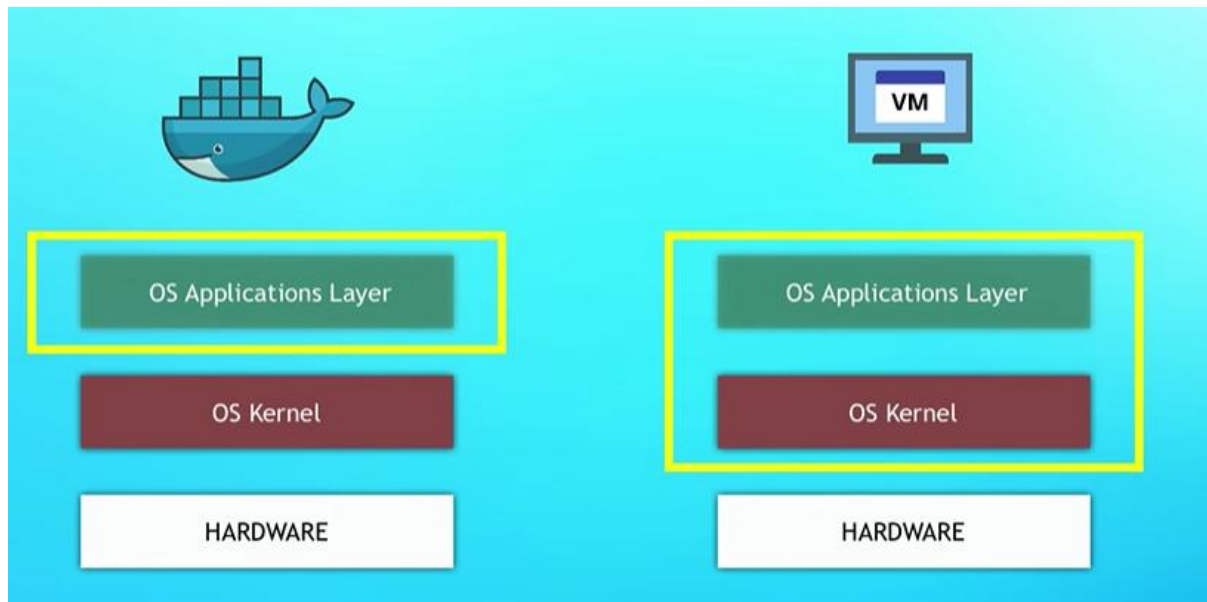


Operating System:

- When we install OS in physical machine.
- Kernel is at the core of every OS.
- Kernel interacts between hardware & software components



(Now the question is, Docker and VM both are virtualization tool what part of the OS do they virtualise???)



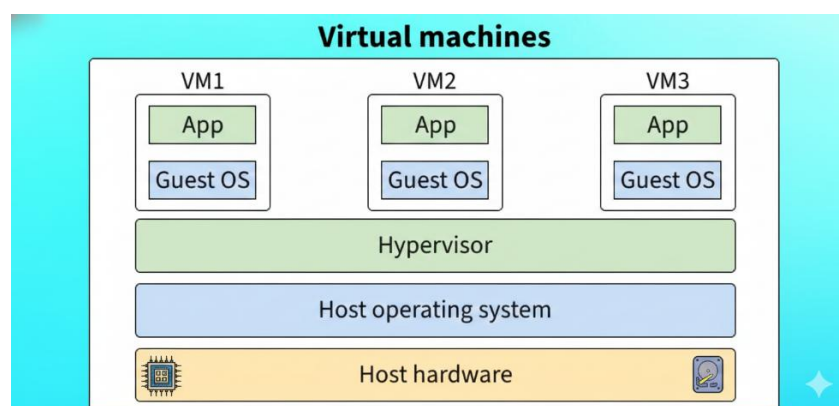
Virtual Machine, Hypervisor and guest OS:

- **Before Virtual Machines:**

Only one application was usually run on one physical server to avoid conflicts.

- **After Virtual Machines:**

A single physical server can run multiple virtual machines. Each virtual machine runs its own application with a separate operating system.



What is a hypervisor??

- A hypervisor is software. (some common hypervisor VMware ESXi, Citrix XenServer, Microsoft Hyper-V)
- A hypervisor is what allows one machine to run multiple virtual machines.
- It's what allocates and controls the sharing of a machine's hardware.

Guest OS and Host OS:

- Guest OS does NOT talk to Host OS
- Guest OS talks to Hypervisor
- Hypervisor gives controlled access to hardware

Case A: Type-2 Hypervisor (Most common)

Example: VirtualBox, VMware Workstation

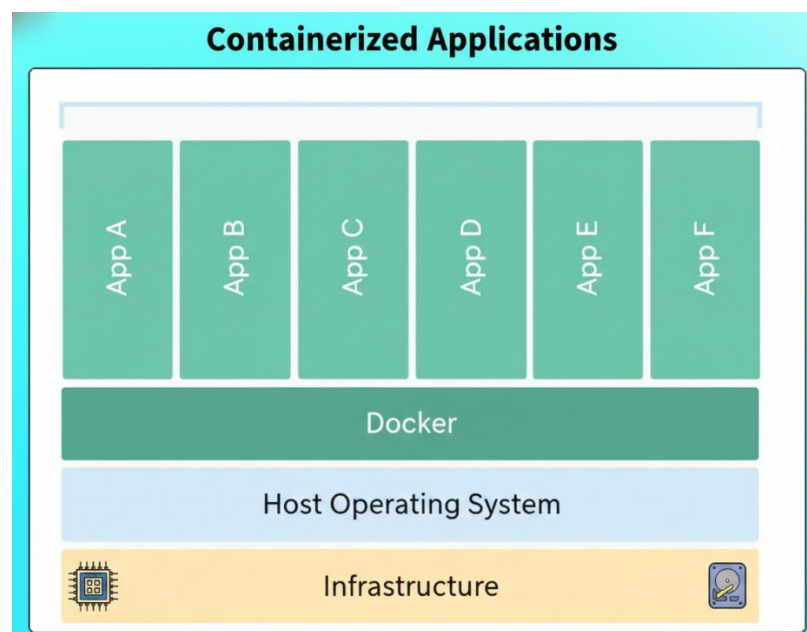
- Hypervisor is installed on top of the Host OS
- It runs like a normal software application

Case B: Type-1 Hypervisor (Servers)

Example: ESXi, Hyper-V (server)

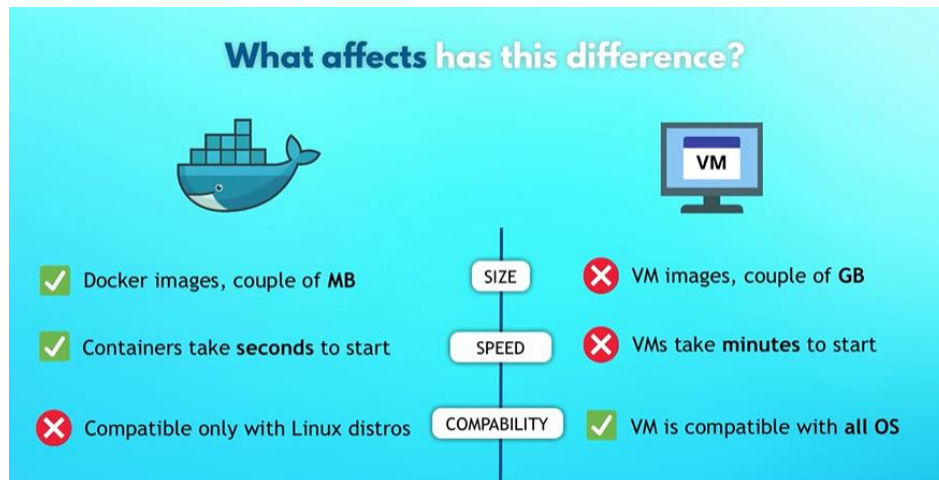
- Hypervisor is installed directly on hardware
- No normal Host OS is required

Docker:



(We already discussed about docker)

Practical – 2 (Docker)



Linux based Docker images, cannot use windows kernel



Most containers are linux based, Docker was originally built for linux OS.

But later Docker actually made an update and developed what's called Docker Desktop for Windows and Mac, which made it possible to run linux based containers on Windows and Mac computers as well.



- Docker Desktop uses a Hypervisor layer with a lightweight Linux distro.

3. Install Docker Locally:

Always best to refer to latest installation guide in the official documentation

Step 1: Go to the Official Docker Website

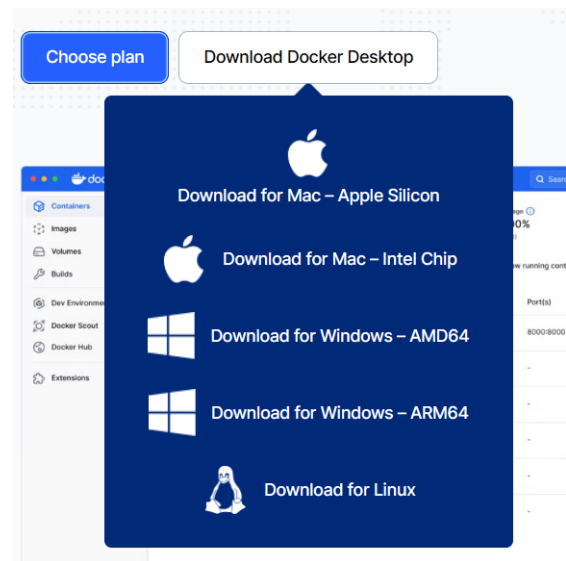
Open your browser and visit [Docker](https://www.docker.com)
(This is the official and safest source)

Step 2:

Click **Download Docker Desktop**

Choose according to your OS:

- **Windows**
- **macOS**
- **Linux**



Step 3: Install Docker Desktop

✅ **For Windows:**

1. Double-click the downloaded .exe file
2. Keep default options checked
3. Click **Install**
4. Restart your system if asked

✅ **For macOS:**

1. Open the downloaded .dmg file
2. Drag Docker icon into **Applications**
3. Open Docker from Applications

✅ **For Linux:** Follow distro-specific steps given on the **Docker** official page(Ubuntu, Debian, Fedora, etc.)

Step 4: Start Docker

- After installation, open Docker Desktop
- Wait until Docker shows “Docker is running”

Step 5: Verify Installation

Open **Command Prompt** / **Terminal** and run: **docker --version**

```
docker --version
```

```
PS C:\Users\ASUS\OneDrive\Desktop> docker --version
Docker version 29.1.5, build 0e6fee6
PS C:\Users\ASUS\OneDrive\Desktop>
```

Docker Desktop includes:

Products inside Docker Desktop

- Docker MCP Toolkit and Catalog
- Docker Model Runner
- Ask Gordon
- Docker Offload
- Docker Engine
- Docker CLI client
- Docker Build
- Docker Compose
- Docker Scout
- Kubernetes

▪ Docker MCP Toolkit and Catalog:

A toolkit that helps Docker tools and AI models communicate using MCP (Model Context Protocol).

Used mainly for AI integrations and advanced workflows.

- **Docker Model Runner**

Runs AI/ML models locally using Docker containers.

Helps developers test and deploy models easily without a complex setup.

- **Ask Gordon**

Docker's AI assistant.

Used to ask questions, get help with Docker commands, errors, and best practices.

- **Docker Offload**

Moves heavy workloads from your local machine to cloud resources.

Improves performance when your system has limited power.

- **Docker Engine:**

A server with a long – running daemon process “dockerd”

Docker Engine is the core software that **enables building, running, and managing Docker containers using images**. It acts as the runtime environment for containers.

- **Docker CLI – Client:**

Command line interface (“Docker”) to **interact with Docker Server**.

Execute Docker commands to start/stop/...containers

It also comes with a GUI Client: To manage your containers and images with a graphical user interface.

- **Docker Compose**

Tool for running multiple containers together.

Uses a **docker-compose.yml** file to define and manage services.

- **Docker Scout**

Security and vulnerability scanning tool.

Checks Docker images for security issues and outdated dependencies.

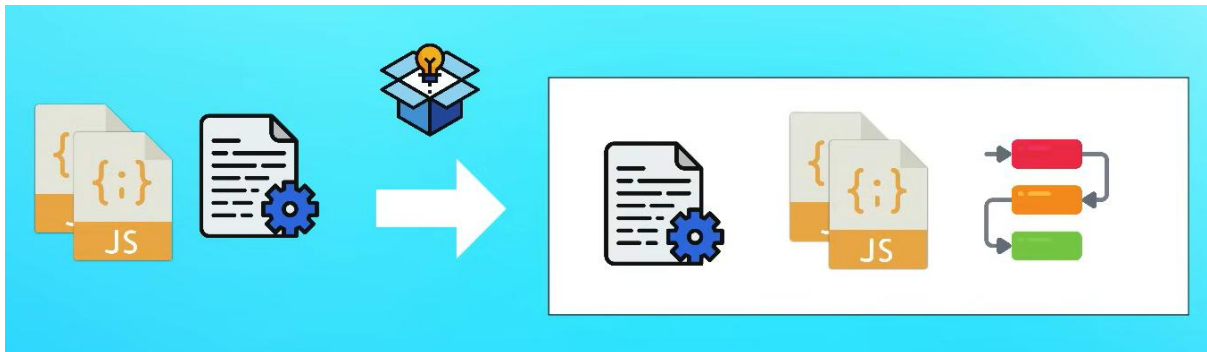
- **Kubernetes**

A container orchestration platform.

Used to manage, scale, and deploy containers in large production environments.

4. Docker Images vs Docker Containers:

Docker allows us to package the application with its environment configuration

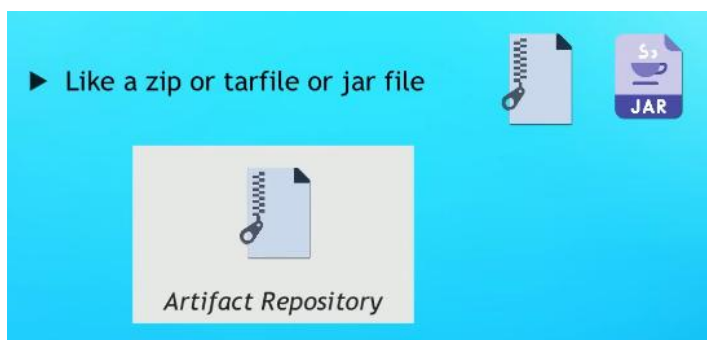


Package everything into an artifact.

Can be easily shared and moved.

(Now, let's understand what an artifact is to make this concept super simple.)

Artifact:



- You can upload zip file to artifact repo. And then download on server or locally when you need it.

You write code → you **build** it → you get a **final file**
(example: app.jar, app.exe, Docker image)

That **final file** is called an **artifact**.

An **artifact repository** is **just a storage place** where these final files are **saved** so you can use them later.

💡 Like this:

- Code = notes you write
- Artifact = final PDF
- Artifact Repository = Google Drive where you save the PDF

A package or artifact that we produce with Docker is called a Docker image

But not like JAR, WAR, etc.,

A Docker Image is an executable application artifact.

Includes app source code, but also complete environment configuration.

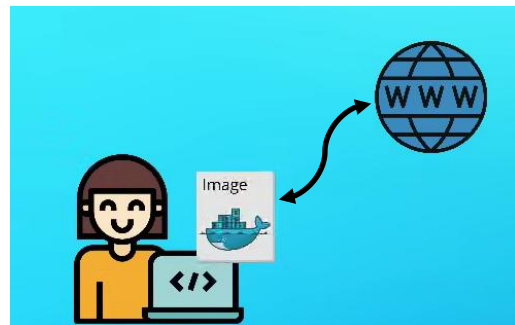
You can add environment variables, create directories, files etc.



The package is called an image, so if that's an image, what is a container then??

Image vs Container:

- We need a place to start the application package. When we download the image to a server or our local computer, we want the application inside it to run.
- When that image is executed on an operating system, the application starts in its pre-configured environment, and this running state is called a **container**.



Analogy:

Docker Image = App on Play Store

- An app on the Play Store is **packaged**
- It has code, settings, and requirements

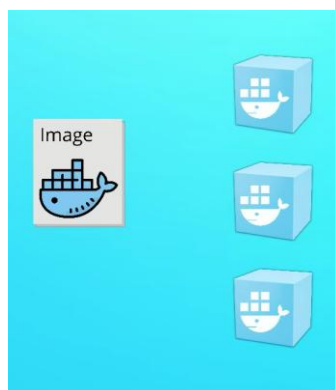
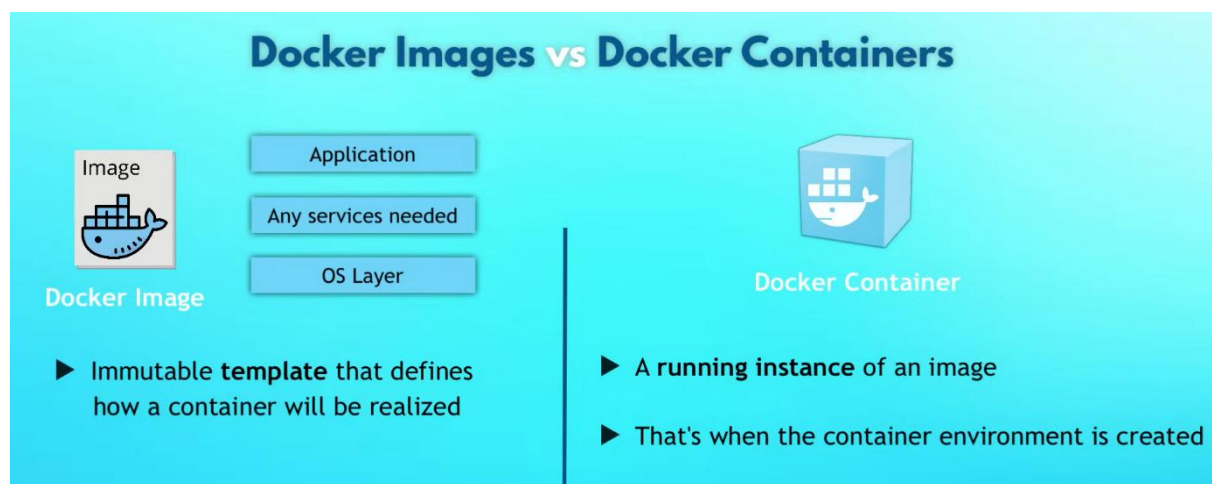
- But it is **not running**

That is exactly what a **Docker image** is.

▶ Container = App Running on Your Phone

- When you **install and open** the app
- The app starts running in its prepared environment

That **running app** is a **container**.



- You can run multiple containers from 1 image

Now you are ready to understand the definition of **Docker Image** and **Docker Container**.

▪ What is a Docker Image?

Definition

A Docker image is a read-only template that contains application code, dependencies, and instructions used to create containers.

▪ What is a Container?

Definition

A container is a lightweight, isolated runtime environment that runs an application along with all required libraries and dependencies.

docker images: show images list which you have pulled

```
PS C:\Users\ASUS\OneDrive\Desktop> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
hello-world:latest	05813aedc15f	25.9kB	9.52kB	U

```
PS C:\Users\ASUS\OneDrive\Desktop> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

```
PS C:\Users\ASUS\OneDrive\Desktop>
```

docker ps = list running containers

5. Docker Registry & Docker Hub

Now it's clear that we run containers from images...

But how do we get these images??

Let's say we want to run a database container or Redis or some log collector service containers. How do we get their Docker Images?

Docker Registries:

That's where Docker Registries come in.



There are ready Docker images available online in image storage or registry. So basically, this is a storage specifically for docker image type of artifacts.

Official images available from applications like Redis, Mongo, Postgres, etc.

Official images are maintained by the software authors or in collaboration with the Docker community.

▪ Image Registry

Definition

An image registry is a storage and distribution system for Docker images.

Docker Hub:

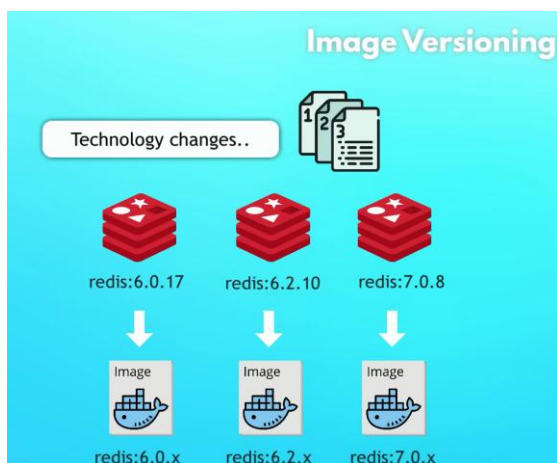


- A storage and distribution system for Docker images.
- Docker hosts one of the biggest Docker Registries, called “Docker Hub”

Docker Hub

Definition

Docker Hub is the default public image registry used by Docker to store, share, and distribute container images.



- Docker Images are versioned
- Different versions are identified by tags

- Docker tags are used to identify images by name

Supported tags and respective Dockerfile links

- [18.1](#) , [18](#) , [latest](#) , [18.1-trixie](#) , [18-trixie](#) , [trixie](#) [↗](#)
- [18.1-bookworm](#) , [18-bookworm](#) , [bookworm](#) [↗](#)
- [18.1-alpine3.23](#) , [18-alpine3.23](#) , [alpine3.23](#) , [18.1-alpine](#) , [18-alpine](#) , [alpine](#) [↗](#)
- [18.1-alpine3.22](#) , [18-alpine3.22](#) , [alpine3.22](#) [↗](#)
- [17.7](#) , [17](#) , [17.7-trixie](#) , [17-trixie](#) [↗](#)
- [17.7-bookworm](#) , [17-bookworm](#) [↗](#)
- [17.7-alpine3.23](#) , [17-alpine3.23](#) , [17.7-alpine](#) , [17-alpine](#) [↗](#)
- [17.7-alpine3.22](#) , [17-alpine3.22](#) [↗](#)

1. Pull command: pull an image form registry

```
docker pull {name}:{tag}
```

```
PS C:\Users\ASUS\OneDrive\Desktop> docker pull nginx:1.29.5
1.29.5: Pulling from library/nginx
bae5a1799a80: Pull complete
7b6cb8ccac7b: Pull complete
46bf3a120c8e: Pull complete
47cd406a84ef: Pull complete
0c8d55a45c0d: Pull complete
4f4efe02d542: Pull complete
f73400a233fd: Pull complete
2e02dba24409: Download complete
a5d78d617315: Download complete
Digest: sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
Status: Downloaded newer image for nginx:1.29.5
docker.io/library/nginx:1.29.5
PS C:\Users\ASUS\OneDrive\Desktop> |
```

```
docker pull {name}
```

```
PS C:\Users\ASUS\OneDrive\Desktop> docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
Digest: sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
PS C:\Users\ASUS\OneDrive\Desktop>
```

Pull the latest image by default

It's use default tag you can see in screen shot.

```
docker images
```

```
PS C:\Users\ASUS\OneDrive\Desktop> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA	In Use
hello-world:latest	05813aedc15f	25.9kB	9.52kB	U	
nginx:1.29.5	341bf0f3ce6c	240MB	65.8MB		
nginx:latest	341bf0f3ce6c	240MB	65.8MB		

```
PS C:\Users\ASUS\OneDrive\Desktop>
```

2. Run: Run a image

```
docker run {name}:{tag} or docker run {name}
```

```
PS C:\Users\ASUS\OneDrive\Desktop> docker run nginx:1.29.5
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to p
erform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-d
efault.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/co
nf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/
conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envs
h
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templat
es.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-process
es.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/02/07 07:35:38 [notice] 1#1: using the "epoll" event method
2026/02/07 07:35:38 [notice] 1#1: nginx/1.29.5
2026/02/07 07:35:38 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/02/07 07:35:38 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/02/07 07:35:38 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/02/07 07:35:38 [notice] 1#1: start worker processes
2026/02/07 07:35:38 [notice] 1#1: start worker process 29
2026/02/07 07:35:38 [notice] 1#1: start worker process 30
2026/02/07 07:35:38 [notice] 1#1: start worker process 31
2026/02/07 07:35:38 [notice] 1#1: start worker process 32
2026/02/07 07:35:38 [notice] 1#1: start worker process 33
2026/02/07 07:35:38 [notice] 1#1: start worker process 34
2026/02/07 07:35:38 [notice] 1#1: start worker process 35
2026/02/07 07:35:38 [notice] 1#1: start worker process 36
2026/02/07 07:35:38 [notice] 1#1: start worker process 37
2026/02/07 07:35:38 [notice] 1#1: start worker process 38
2026/02/07 07:35:38 [notice] 1#1: start worker process 39
2026/02/07 07:35:38 [notice] 1#1: start worker process 40
2026/02/07 07:35:38 [notice] 1#1: start worker process 41
2026/02/07 07:35:38 [notice] 1#1: start worker process 42
2026/02/07 07:35:38 [notice] 1#1: start worker process 43
2026/02/07 07:35:38 [notice] 1#1: start worker process 44
```

You can see it's blocking a terminal, look at the end line.

Note:

Docker run command

Creates a new container

Doesn't re -use previous container.

Practical – 2 (Docker)

```
PS C:\Users\ASUS> docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED         STATUS
08ae15f2db42   nginx:1.29.5  "/docker-entrypoint..."  5 minutes ago   Up 5
minutes      80/tcp     heuristic_knuth
PS C:\Users\ASUS>
```



How to Exit: Ctrl + c

If you do Ctrl + c the container exits, and the process actually dies.

Let's see using the docker ps command

```
PS C:\Users\ASUS> docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED         STATUS         PORTS      NAMES
PS C:\Users\ASUS> |
```

But we can start a container in the background without it blocking the terminal by adding flag

`docker run -it -d <image_name>`

```
PS C:\Users\ASUS> docker run -d nginx:1.29.5
8f284ebf268955ae7ce64d591e102a80400c7677e580deb2bf2b481fe3bccc51
PS C:\Users\ASUS> docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED         STATUS
US            PORTS      NAMES
8f284ebf2689   nginx:1.29.5  "/docker-entrypoint..."  16 minutes ago   Up 1
6 minutes      80/tcp     beautiful_shaw
PS C:\Users\ASUS>
```

now, you can see it's not blocking the terminal.

-d or --detach = run container in background and prints the container ID.

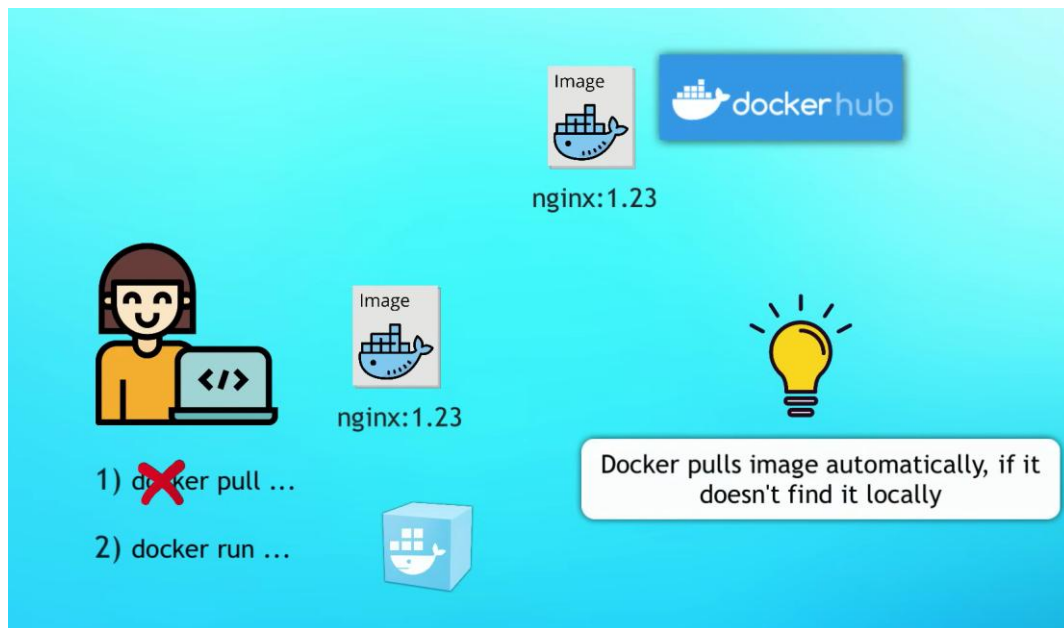
When you run a container in detached mode (-d), Docker starts it in the background you won't see logs on your terminal. Here's how you check which service is running and what it's doing.

3. Log: View logs of the running service

```
docker logs <container_id or name>
```

```
PS C:\Users\ASUS> docker logs 8f284ebf268955ae7ce64d591e102a80400c7677e580debf2bfb481fe3bccc51
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/02/07 11:32:07 [notice] 1#1: using the "epoll" event method
2026/02/07 11:32:07 [notice] 1#1: nginx/1.29.5
2026/02/07 11:32:07 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/02/07 11:32:07 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/02/07 11:32:07 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/02/07 11:32:07 [notice] 1#1: start worker processes
2026/02/07 11:32:07 [notice] 1#1: start worker process 29
2026/02/07 11:32:07 [notice] 1#1: start worker process 30
2026/02/07 11:32:07 [notice] 1#1: start worker process 31
2026/02/07 11:32:07 [notice] 1#1: start worker process 32
2026/02/07 11:32:07 [notice] 1#1: start worker process 33
2026/02/07 11:32:07 [notice] 1#1: start worker process 34
2026/02/07 11:32:07 [notice] 1#1: start worker process 35
2026/02/07 11:32:07 [notice] 1#1: start worker process 36
2026/02/07 11:32:07 [notice] 1#1: start worker process 37
2026/02/07 11:32:07 [notice] 1#1: start worker process 38
2026/02/07 11:32:07 [notice] 1#1: start worker process 39
2026/02/07 11:32:07 [notice] 1#1: start worker process 40
2026/02/07 11:32:07 [notice] 1#1: start worker process 41
2026/02/07 11:32:07 [notice] 1#1: start worker process 42
2026/02/07 11:32:07 [notice] 1#1: start worker process 43
2026/02/07 11:32:07 [notice] 1#1: start worker process 44
PS C:\Users\ASUS>
```

NOTE: (Both in one command)



```
PS C:\Users\ASUS> docker run alpine
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
589002ba0eae: Pull complete
9e595aac14e0: Download complete
caa817ad3aea: Download complete
Digest: sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c3f659
Status: Downloaded newer image for alpine:latest
PS C:\Users\ASUS> |
```

6. Port Binding:

Container Port

- The port inside the container
- Where the application actually runs
- Known only inside Docker

Example:

A web app runs on port 80 *inside* the container.

Local (Host) Port

- The port on your computer/server
- What you use in browser or API calls
- Maps traffic to the container port

Example:

You open `http://localhost:8080` in your browser.

Port binding means:

Bind the container's port to the host's port to make the service available to the outside world.

Why port binding is needed?

Containers are isolated.

Apps inside them are not reachable unless you bind ports.

Without port binding ❌

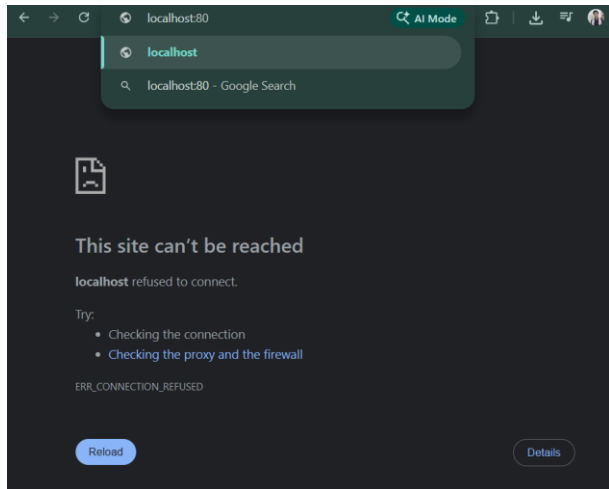
👉 You cannot open the app in browser.

With port binding ✅

👉 You can access it via localhost.

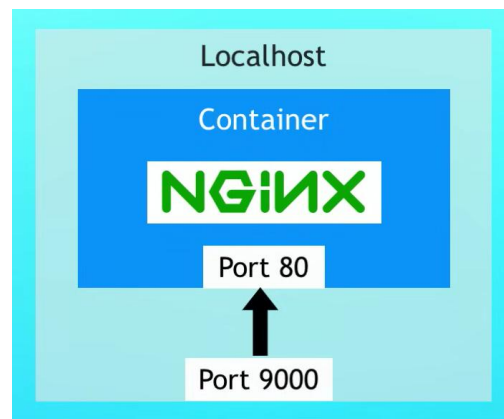
Practical – 2 (Docker)

```
PS C:\Users\ASUS> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
1350b5231f1b   nginx:latest "/docker-entrypoint...." 3 seconds ago Up 3 seconds 80/tcp       gallant_faraday
PS C:\Users\ASUS>
```



Let's bind container port 80 to our local port 8080 or 9000 or

Let's first stop running container



4. Stop: to stop the running container

```
docker stop <container id>
```

```
PS C:\Users\ASUS> docker stop 1350b5231f1b
1350b5231f1b
PS C:\Users\ASUS> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
PS C:\Users\ASUS> |
```

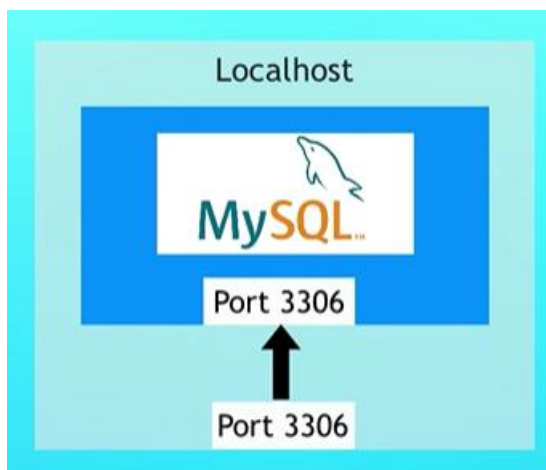
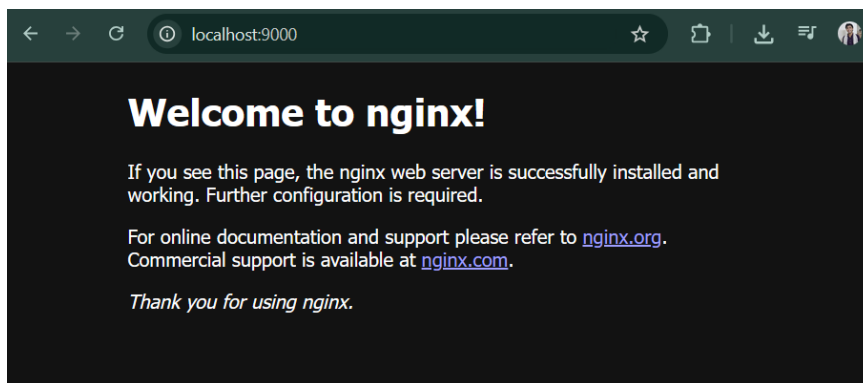
5. Port Binding:

```
docker run -d -p {host_port}:{container_port} <container name>
```

```
PS C:\Users\ASUS> docker run -d -p 9000:80 nginx
9dd90c9f90f88f729560797d7ade5b955f2bddaef2ece06d3cc6e10f33c01ede
PS C:\Users\ASUS> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS
PORTS         NAMES
9dd90c9f90f8   nginx    "/docker-entrypoint...." 14 seconds ago Up 13 sec
0.0.0.0:9000->80/tcp, [::]:9000->80/tcp    dazzling_lamport
PS C:\Users\ASUS> |
```

Only 1 service can run on a specific port on the host

So, e.g. only 1 service can run on port 9000



Standard to use the same port on your host as container is using

As we know, the docker run command creates a new container every time it is executed. However, these containers do not appear when we run the docker ps command because it shows only running containers. To see how many containers have been created so far, we can use the docker ps -a command.

6. List of all created containers:

```
docker ps -a
```

```
PS C:\Users\ASUS> docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED
STATUS
NAMES
9dd90c9f90f8   nginx         "/docker-entrypoint..." 23 minutes ago
Up 23 minutes
dazzling_lampor
1350b5231f1b   nginx:latest  "/docker-entrypoint..." 55 minutes ago
Exited (0) 37 minutes ago
gallant_faraday
b42603fe2cb5   nginx:latest  "/docker-entrypoint..." 55 minutes ago
Exited (0) 55 minutes ago
admiring_cori
02ec10bfd64b   alpine        "/bin/sh"                About an hour ago
Exited (0) About an hour ago
vigorous_maxwell
8f284ebf2689   nginx:1.29.5  "/docker-entrypoint..." 2 hours ago
Exited (0) About an hour ago
beautiful_shaw
dec82094c59a   hello-world:latest "/hello"                  6 hours ago
Exited (0) 6 hours ago
gifted_easley
ad05efa1bb6e   hello-world    "/hello"                  6 hours ago
Exited (0) 6 hours ago
dazzling_saha
08ae15f2db42   nginx:1.29.5  "/docker-entrypoint..." 6 hours ago
Exited (0) 6 hours ago
heuristic_knuth
57a7b756d6eb   hello-world    "/hello"                  4 days ago
Exited (0) 4 days ago
distracted_greider
PS C:\Users\ASUS> |
```

Showing up and Exited containers.

We can use these exited containers without creating a new container.

Cuz they still exist.

7. Start: is used to start an *existing* container

```
docker start <id>
```

```
PS C:\Users\ASUS> docker start dec82094c59a
dec82094c59a
PS C:\Users\ASUS> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED NAMES	STATUS
9dd90c9f90f8	nginx	"/docker-entrypoint..."	36 minutes ago	Up 36 min
utes 0.0.0.0:9000->80/tcp, [::]:9000->80/tcp			dazzling_lamport	

```
PS C:\Users\ASUS> |
```

You can use name as well.

8. Name: Is to assign a name to container

```
docker run --name <give name > <docker name>
```

```
PS C:\Users\ASUS> docker run --name bunny -d -p 9000:80 nginx
0e8a95bac7c3cf73364b2dc66ba5ec0d045c42534581cfe60d60ddd4d0e0219a
PS C:\Users\ASUS> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED NAMES	STATUS
0e8a95bac7c3	nginx	"/docker-entrypoint..."	13 seconds ago	Up 12 sec
onds 0.0.0.0:9000->80/tcp, [::]:9000->80/tcp			bunny	

```
PS C:\Users\ASUS> |
```

7. Public and Private Docker Registries & Dockerfile:

- Largest public registry
- Anyone can search and download Docker images

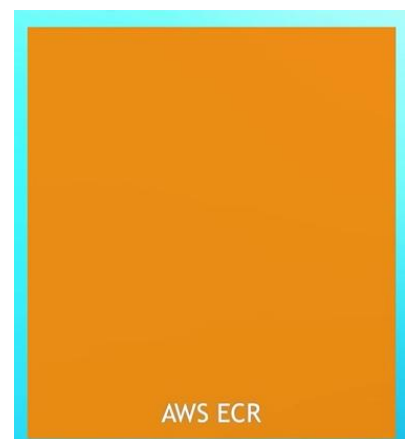


- You need to authenticate before accessing the registry.
- All big cloud providers offer private registries: Amazon ECR, Google Registry, etc.
- Nexus (popular artifact repository manager)
- Docker Hub!

Docker Registry vs Docker Repository:

Docker Registry:

- A service providing storage
- Can be hosted by a third party, like AWS, or by Yourself
- Collection of repositories



Docker Repository:

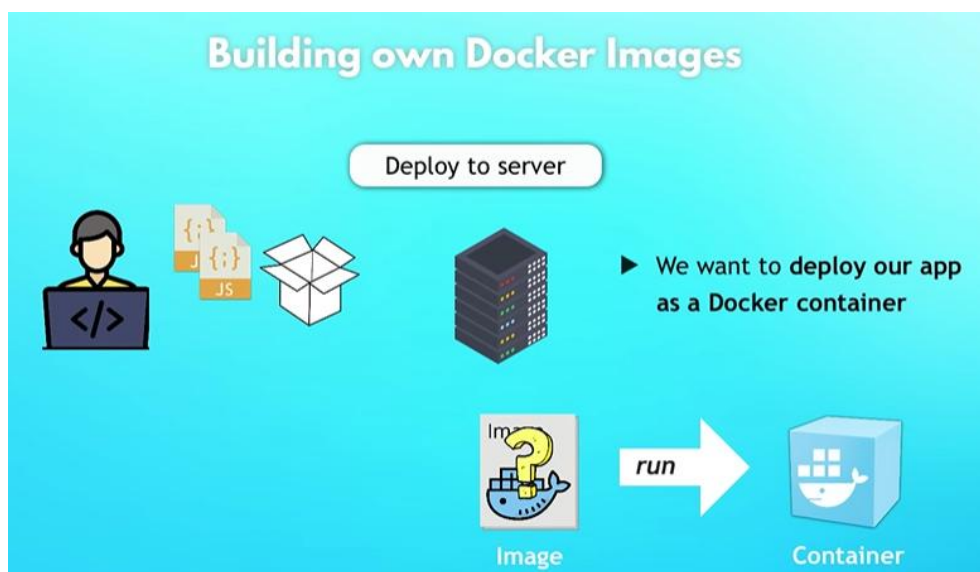
- Collection of related images with the same name but different versions

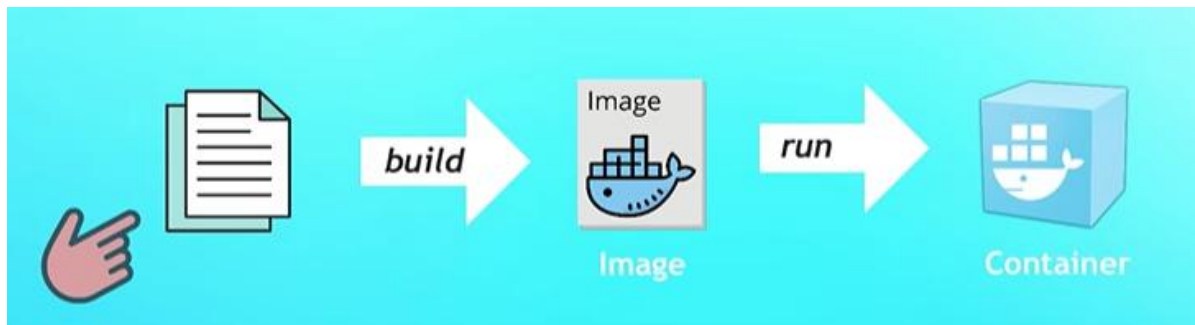


Docker Hub is a registry

On Docker Hub, you can host private or public repositories for your applications

Companies create custom images for their applications.





We need to create a “definition” of how to build an image from our application.

Here, Dockerfile comes into the scene.

Dockerfile (MOST IMPORTANT)

Definition

A Dockerfile is a text file containing **step-by-step instructions to build a Docker image**.

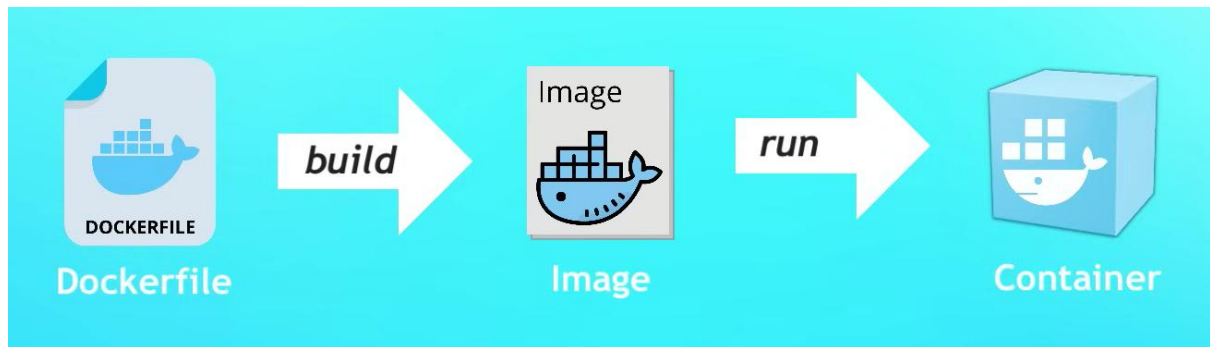
Structure of Dockerfile

- Dockerfiles start from a parent image or “base image”
- It’s a Docker image that your image is based on.

(You chose the base image, depending on which tools you need to have available)



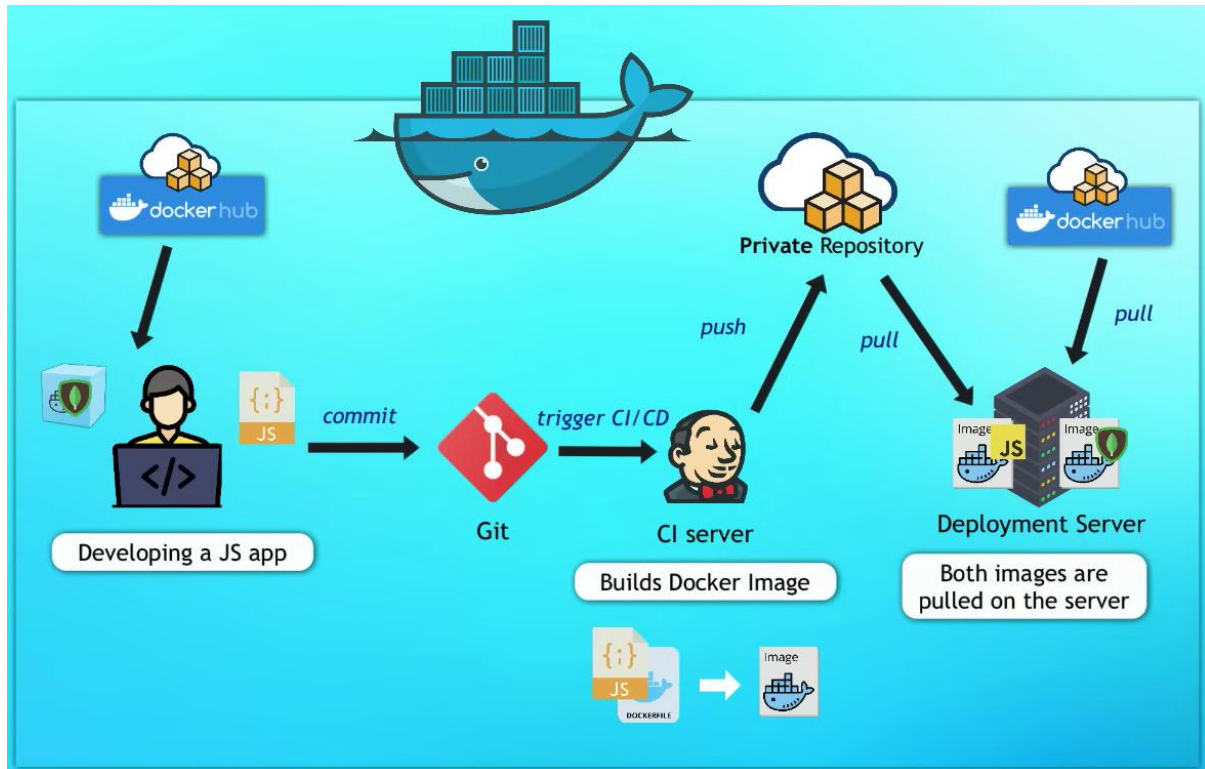
Practical – 2 (Docker)



You can watch any YouTube video for this concept. For better and fast understanding.

8. END part:

How Docker fits in the complete development and deployment process?



PART-2: COMMANDS (Clg practical)

1. Docker –Version:

```
PS C:\Users\ASUS> docker --version
Docker version 29.1.5, build 0e6fee6
PS C:\Users\ASUS>
```

2. docker info

```
PS C:\Users\ASUS> docker info
Client:
Version:      29.1.5
Context:      desktop-linux
Debug Mode:  false
Plugins:
ai: Docker AI Agent - Ask Gordon (Docker Inc.)
   Version:   v1.17.1
   Path:      C:\Program Files\Docker\cli-plugins\docker-ai.exe
buildx: Docker Buildx (Docker Inc.)
   Version:   v0.30.1-desktop.2
   Path:      C:\Program Files\Docker\cli-plugins\docker-buildx.exe
compose: Docker Compose (Docker Inc.)
   Version:   v5.0.1
   Path:      C:\Program Files\Docker\cli-plugins\docker-compose.exe
debug: Get a shell into any image or container (Docker Inc.)
   Version:   0.0.47
   Path:      C:\Program Files\Docker\cli-plugins\docker-debug.exe
desktop: Docker Desktop commands (Docker Inc.)
   Version:   v0.2.0
   Path:      C:\Program Files\Docker\cli-plugins\docker-desktop.exe
extension: Manages Docker extensions (Docker Inc.)
   Version:   v0.2.31
   Path:      C:\Program Files\Docker\cli-plugins\docker-extension.exe
init: Creates Docker-related starter files for your project (Docker Inc.)
   Version:   v1.4.0
   Path:      C:\Program Files\Docker\cli-plugins\docker-init.exe
```

3. docker pull alpine

```
PS C:\Users\ASUS> docker pull alpine
Using default tag: latest
latest: Pulling from library/alpine
Digest: sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c3f659
Status: Image is up to date for alpine:latest
docker.io/library/alpine:latest
PS C:\Users\ASUS>
```

4. docker run -it -d alpine

```
PS C:\Users\ASUS> docker run -it -d alpine
3933e9b403c13bcacf7bdc0ad7e31b046fe57522db35a232f3d67a021d4eb8e9
PS C:\Users\ASUS> docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
3933e9b403c1   alpine   "/bin/sh" 5 seconds ago  Up 4 seconds           gallant_ride
PS C:\Users\ASUS>
```

-i → interactive

Keeps **STDIN open** (input channel open)

-t → terminal

Allocates a **TTY (terminal)**

5. docker exec -it <container_id> sh

```
PS C:\Users\ASUS> docker exec -it 3933e9b403c1 sh
/ #
```

6. exit

```
PS C:\Users\ASUS> docker exec -it 3933e9b403c1 sh
/ # exit
PS C:\Users\ASUS>
```

7. docker start <container ID>

```
PS C:\Users\ASUS> docker start 3933e9b403c1
3933e9b403c1
PS C:\Users\ASUS> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS
NAMES
3933e9b403c1   alpine    "/bin/sh"               11 minutes ago Up 2 seconds
gallant_ride
PS C:\Users\ASUS>
```

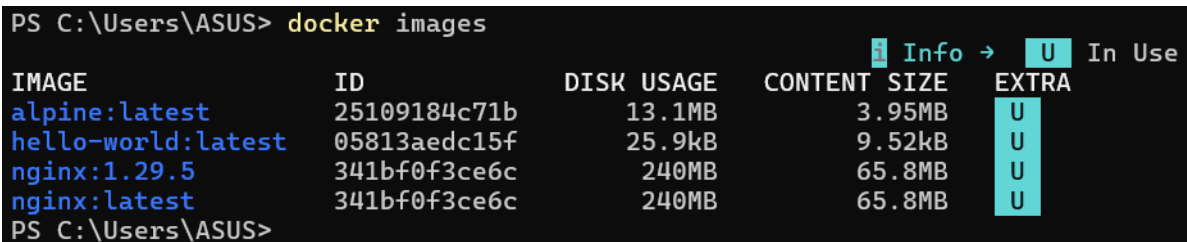
8. docker stop <container ID>

```
PS C:\Users\ASUS> docker stop 3933e9b403c1
3933e9b403c1
PS C:\Users\ASUS> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
PS C:\Users\ASUS>
```

9. docker kill <container ID>

```
PS C:\Users\ASUS> docker start 3933e9b403c1
3933e9b403c1
PS C:\Users\ASUS> docker kill 3933e9b403c1
3933e9b403c1
PS C:\Users\ASUS> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
PS C:\Users\ASUS>
```

10. docker images

```
PS C:\Users\ASUS> docker images


| IMAGE              | ID           | DISK USAGE | CONTENT SIZE | EXTRA | In Use |
|--------------------|--------------|------------|--------------|-------|--------|
| alpine:latest      | 25109184c71b | 13.1MB     | 3.95MB       | U     |        |
| hello-world:latest | 05813aedc15f | 25.9kB     | 9.52kB       | U     |        |
| nginx:1.29.5       | 341bf0f3ce6c | 240MB      | 65.8MB       | U     |        |
| nginx:latest       | 341bf0f3ce6c | 240MB      | 65.8MB       | U     |        |


PS C:\Users\ASUS>
```

CONTAINER RESOURCE ALLOCATION:

11. `docker run -it --name one_gb --memory="1g" alpine`

```
PS C:\Users\ASUS> docker run -it --name onegb --memory="1g" alpine / #
```

`--memory="1g"` → limits RAM to 1GB

12. `docker stats`

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
0a1cc5f4a034	one_gb	0.00%	508KiB / 1GiB	0.05%	1.17kB / 126B	28.7kB / 0B	1

13. `docker inspect <container id>`

`docker inspect` is used to view complete metadata and configuration details of a Docker container or image.

```
PS C:\Users\ASUS> docker inspect one_gb
[
  {
    "Id": "0a1cc5f4a03483466116c80daf15f53074166142216eb6cb0c1b35cb226fa447",
    "Created": "2026-02-07T16:26:41.768208376Z",
    "Path": "/bin/sh",
    "Args": [],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 2711,
      "ExitCode": 0,
      "Error": "",
      "StartedAt": "2026-02-07T16:30:25.996821702Z",
      "FinishedAt": "2026-02-07T16:27:05.888248426Z"
    },
    "Image": "sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c3f659",
    "ResolvConfPath": "/var/lib/docker/containers/0a1cc5f4a03483466116c80daf15f53074166142216eb6cb0c1b35cb226fa447/resolv.conf",
    "HostnamePath": "/var/lib/docker/containers/0a1cc5f4a03483466116c80daf15f53074166142216eb6cb0c1b35cb226fa447/hostname",
    "HostsPath": "/var/lib/docker/containers/0a1cc5f4a03483466116c80daf15f53074166142216eb6cb0c1b35cb226fa447/hosts",
    "LogPath": "/var/lib/docker/containers/0a1cc5f4a03483466116c80daf15f53074166142216eb6cb0c1b35cb226fa447/0a1cc5f4a03483466116c80daf15f53074166142216eb6cb0c1b35cb226fa447-json.log",
    "Name": "/one_gb",
    "RestartCount": 0,
    "Driver": "overlayfs",
    "Platform": "linux",
  }
]
```

14. `docker run -it --memory="1g" --memory-swap="2g" alpine`

```
PS C:\Users\ASUS> docker run -it --memory="1g" --memory-swap="2g" alpine
/ # exit
PS C:\Users\ASUS> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS
NAMES
0a1cc5f4a034   alpine    "/bin/sh"               12 minutes ago Up 8 minutes
one_gb
```

Explanation:

RAM limit = **1GB** (1073741824 bytes $\approx$ 1GB RAM)

Total RAM + Swap = **2GB** (2147483648 bytes $\approx$ 2GB RAM + swap)

DOCKER VOLUME COMMANDS & DEMONSTRATION:

What is a Docker Volume?

A Docker volume is a safe place on your host machine where Docker stores container data permanently.

☞ Even if the container stops, restarts, or is deleted, the data in the volume does NOT disappear.

15. `docker volume create data`

Creates a new volume named **data**.

```
PS C:\Users\ASUS\OneDrive\Desktop> docker volume create data
data
PS C:\Users\ASUS\OneDrive\Desktop>
```

16. `docker volume ls`

Displays all available docker volumes.

```
PS C:\Users\ASUS\OneDrive\Desktop> docker volume ls
DRIVER      VOLUME NAME
local      data
PS C:\Users\ASUS\OneDrive\Desktop>
```

17. `docker volume inspect data`

Shows volume metadata, mount path, driver, and scope.

```
PS C:\Users\ASUS\OneDrive\Desktop> docker volume inspect data
[
  {
    "CreatedAt": "2026-02-08T04:45:08Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/data/_data",
    "Name": "data",
    "Options": null,
    "Scope": "local"
  }
]
PS C:\Users\ASUS\OneDrive\Desktop>
```

18. `docker run -it --name example1 --mount source=data,destination=/data alpine`

```
PS C:\Users\ASUS\OneDrive\Desktop> docker run -it --name=example1
--mount source=data,destination=/data alpine
/ #
```

--mount source=data,destination=/data

This is the **most important part** 🙌

It means:

- `source=data`
 - 🙌 Docker **volume name** = data
 - 🙌 If it doesn't exist, Docker **creates it automatically**
 - `destination=/data`
 - 🙌 Inside the container, the volume appears at /data
- 📌 Anything written to /data goes into the volume.

19. Sharing Volume B/w Containers

```
PS C:\Users\ASUS\OneDrive\Desktop> docker run -it --name example3
--mount source=data,destination=/data alpine
/ # exit
PS C:\Users\ASUS\OneDrive\Desktop> docker run -it --name example2
--mount source=data,destination=/data alpine
/ # exit
PS C:\Users\ASUS\OneDrive\Desktop>
```


20. Read – Only Volume Sharing

example1 owns the volume , example4 can **only read**, not write

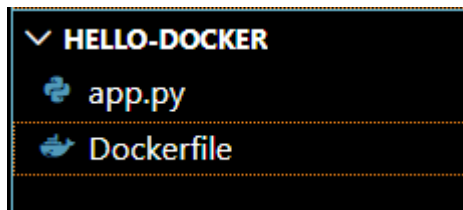
```
PS C:\Users\ASUS\OneDrive\Desktop> docker run -it --name example4 --volumes-  
from example1:ro alpine  
\# # cd data  
/data # ls  
/data # touch test.txt  
touch: test.txt: Read-only file system  
/data #
```

HELLO DOCKER WITH PYTHON(USING ALPINE)

What we gone do:

Create a simple Python program and Dockerize it using Alpine Linux image.

Project Structure:



Step 1: creted project folder

```
PS C:\Users\ASUS\OneDrive\Desktop> mkdir HelloDocker

Directory: C:\Users\ASUS\OneDrive\Desktop

Mode                LastWriteTime         Length Name
----                -
d-----           08-02-2026         10:54         HelloDocker
```

Step 2: Create app.py

```
PS C:\Users\ASUS\OneDrive\Desktop\HelloDocker> notepad app.py
PS C:\Users\ASUS\OneDrive\Desktop\HelloDocker> dir

Directory: C:\Users\ASUS\OneDrive\Desktop\HelloDocker

Mode                LastWriteTime         Length Name
----                -
-a---l           08-02-2026         10:57          26 app.py

PS C:\Users\ASUS\OneDrive\Desktop\HelloDocker> |
```

Code inside app.py: `print("Hello Docker 🚀")`

Step 3: Created Dockerfile

```
Dockerfile
1 FROM python:3.12-alpine
2 WORKDIR /app
3 COPY app.py .
4 CMD ["python", "app.py"]
```

Step 4: Build Docker

```
PS C:\Users\ASUS\OneDrive\Desktop\HelloDocker> docker build -t hello-python .
[+] Building 11.5s (9/9) FINISHED          docker:desktop-linux
=> [internal] load build definition from Dockerfile 0.1s
=> => transferring dockerfile: 123B 0.0s
=> [internal] load metadata for docker.io/library/python:3.12-alpine 3.9s
=> [auth] library/python:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [1/3] FROM docker.io/library/python:3.12-alpine@sha256:2d91681153 6.2s
=> => resolve docker.io/library/python:3.12-alpine@sha256:2d91681153 0.0s
=> => sha256:77e59d578b52ba7be2dcd4414d9ef235af6ea92f56d 247B / 247B 0.8s
=> => sha256:4e18f69996e78732b37aff52d0d056f1513df 13.74MB / 13.74MB 5.4s
=> => sha256:aca07fe076f12b62834e15448c6ec185884 460.94kB / 460.94kB 1.3s
=> => extracting sha256:aca07fe076f12b62834e15448c6ec1858845fe8029d7 0.3s
=> => extracting sha256:4e18f69996e78732b37aff52d0d056f1513df14ab58 0.5s
=> => extracting sha256:77e59d578b52ba7be2dcd4414d9ef235af6ea92f56d5 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 59B 0.0s
=> [2/3] WORKDIR /app 0.1s
=> [3/3] COPY app.py . 0.1s
=> exporting to image 0.6s
=> => exporting layers 0.2s
=> => exporting manifest sha256:c35e0f0d92a98d60005eaf26da14f12806ff 0.0s
=> => exporting config sha256:318f6548c13ae8c380db05ecb56aacecf1c43d 0.0s
=> => exporting attestation manifest sha256:f4b2013b54fe3dcb3040c00f 0.1s
=> => exporting manifest list sha256:94f8fdc5fc5fbc83f335349835f6f0a 0.0s
=> => naming to docker.io/library/hello-python:latest 0.0s
=> => unpacking to docker.io/library/hello-python:latest 0.1s
PS C:\Users\ASUS\OneDrive\Desktop\HelloDocker>
```

docker build → create image

-t hello-python → image name

. → current folder (uses Dockerfile here)

Step 5: Check image is created or not

```
PS C:\Users\ASUS\OneDrive\Desktop\HelloDocker> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	Info →	U	In Use
alpine:latest	25109184c71b	13.1MB	3.95MB		U	
hello-python:latest	94f8fdc5fc5f	74.3MB	18.1MB		U	
hello-world:latest	05813aedc15f	25.9kB	9.52kB		U	
nx:1.29.5	341bf0f3ce6c	240MB	65.8MB		U	
nx:latest	341bf0f3ce6c	240MB	65.8MB		U	

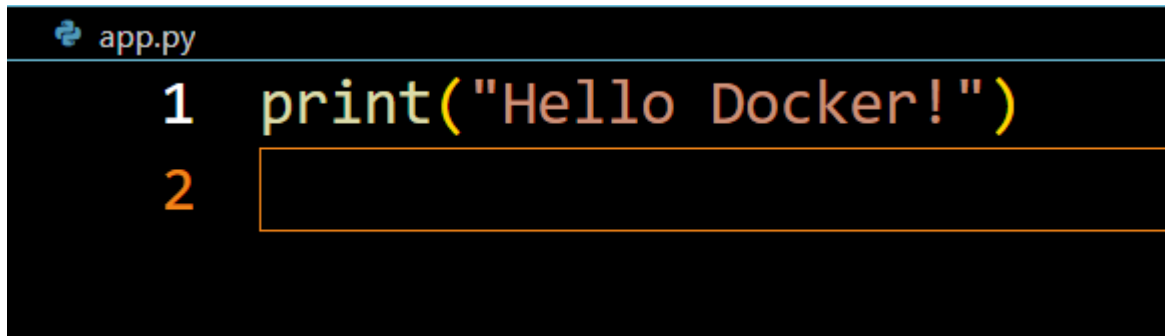
```
PS C:\Users\ASUS\OneDrive\Desktop\HelloDocker>
```



Step 6: Run Docker

```
PS C:\Users\ASUS\OneDrive\Desktop\HelloDocker> docker run hello-python
Hello Docker 🚀
PS C:\Users\ASUS\OneDrive\Desktop\HelloDocker>
```

Step 1: create app.py

A screenshot of a code editor with a dark background. The file name 'app.py' is visible in the top left corner. The code consists of two lines: line 1 contains 'print("Hello Docker!")' and line 2 is an empty line with a cursor. The line numbers '1' and '2' are highlighted in orange on the left side of the editor.

```
app.py
1 print("Hello Docker!")
2
```

Step 2: create dockerfile

- python:3.12-alpine → lightweight Alpine Linux with Python
- WORKDIR /app → working directory inside container
- COPY app.py . → copy program
- CMD → run program